

Програмування мовою C++. Оператори керування: оператор умовного переходу if та оператор-перемикач switch. Параметричний оператор циклу for ОПЕРАТОРИ ЦИКЛУ .CONTINUE/BREAK/

1. Оператори умовного переходу (if)

Оператор if дозволяє виконувати різні гілки програми залежно від істинності умови.

1.1. Повна форма

```
if (умова) {  
    // інструкції  
} else {  
    // інструкції  
}
```

1.2. Скорочена форма

```
if (умова)  
    оператор;
```

Використовується, коли гілка «else» не потрібна.

1.3. Ланцюжок if – else if – else

Для багатьох умов:

```
if (x < 0) ...  
else if (x <= 10) ...  
else ...
```

1.4. Вкладені if та проблема "висячого else"

else належить до найближчого if.

Щоб уникнути плутанини — завжди використовувати { }.

1.5. Типові помилки

- = замість ==
- зайва крапка з комою після if (cond);

2. Оператор вибору switch

Використовується, коли треба вибрати одну дію з багатьох за значенням змінної.

2.1. Загальна форма

```
switch (вираз) {  
    case значення1:  
        оператори;  
        break;  
    case значення2:  
        оператори;  
        break;  
    ...  
    default:  
        оператори;  
}
```

2.2. Особливості

- порівнює *лише з цілими або символами*
- break перериває виконання і виходить зі switch
- якщо break немає — виконується "провалювання"

2.3. Приклад

```
switch (day) {  
    case 6:  
    case 7:  
        fDayOff = true;  
        break;  
    default:  
        fDayOff = false;  
}
```

3. Цикли в C++

Усі цикли повторюють набір команд кілька разів, поки істинна певна умова.

3.1. Цикл for

Цикл з лічильником.

```
for (ініціалізація; умова; крок) {  
    ...  
}
```

Приклад:

```
for (int i = 1; i <= n; i++)  
    sum += 1.0 / i;
```

Може мати кілька лічильників:

```
for (i = 1, j = 9; i <= 9; i++, j--)
```

3.2. Цикл **while** (з передумовою)

Перевіряє умову *перед* виконанням тіла.

```
while (умова) {  
    ...  
}
```

Приклад — підрахунок цифр у числі:

```
while (t > 0) {  
    if (t % 10 == 3) k++;  
    t /= 10;  
}
```

3.3. Цикл **do...while** (з постумовою)

Виконується *мінімум один раз*.

```
do {  
    ...  
} while (умова);
```

4. Оператори керування циклом: **break** і **continue**

4.1. **break**

- припиняє виконання циклу достроково
- працює у: for, while, do...while, switch

Приклад:

```
for (;;) {
    cin >> x;
    if (x == 0) break;
    sum += x;
}
```

4.2. continue

- переходить до наступної ітерації
- пропускає частину тіла циклу

```
for (int i = 0; i < 5; i++) {
    if (i == 3) continue;
    cout << i << endl;
}
```

4.3. У while / do-while треба збільшувати лічильник вручну!

```
while (i < 5) {
    if (i == 3) {
        i++;
        continue;
    }
    i++;
}
```

4.4. Вкладені цикли

break виходить лише з *внутрішнього* циклу.

5. Приклади застосування

5.1. Сума введених чисел до введення 0

```
for (;;) {
    cin >> x;
    if (x == 0) break;
```

```
    sum += x;  
}
```

5.2. Обчислення π з заданою точністю

Метод Лейбніца, вихід через break, коли досягнута точність.

5.3. Сума чисел у проміжку [-5; 5] до зустрічі 0

```
do {  
    cin >> number;  
    if (number < -5 || number > 5) continue;  
    sum += number;  
} while (number != 0);
```

6. Інші оператори потоку керування

6.1. goto

- здійснює безумовний перехід до мітки
- небажаний у реальному коді
- дозволений у виняткових випадках (наприклад, вихід з вкладених циклів)

6.2. return

- завершує роботу функції
- може також використовуватися для виходу із switch

7. Підсумкова схема

Умовні оператори

- if, if/else, else if
- switch

Цикли

- for
- while
- do...while

Оператори керування циклом

- `break` — вийти з циклу
- `continue` — перейти до наступної ітерації

Перехідні оператори

- `goto` — не рекомендується
- `return` — завершити функцію